Дискретный анализ

Вайб-лекция 7: суффиксный массив, LCP и FM-индекс

Андрей Калинин

Дискретный анализ, 2026/27

11 ноября 2026

Конкурс по сортировке: таблица на 10 ноября

Место	Ник	Очки	Быстрее `std::sort` в…
1	jubik666	**19,7**	от 3,8 (small100) до 52 (sorted)
2	DanSivet	**16,7**	от 1,9 (small100) до 56 (organpipe)
3	kinyex	3,0	от 0,9 до 26
4–15	12 ников	0,46	шаблон без изменений

Срок сдачи: **6 декабря, 23:59 МСК**.

Очки — во сколько раз быстрее `std::sort` в среднем по 14 наборам.



2026/contest/sorting/leaderboard.md

Раунд practice 2026-11-10 08:51 UTC, набор full, лучшее из 5 запусков.

Где мы и куда идём

Прошлый раз: один текст, много запросов — готовим **текст**. Суффиксное дерево даёт всё: вхождения за $O(m + \mathrm{occ})$, повторы, общую подстроку, MUM. Цена — память: 37 байт на букву у MUMmer 1999.

Что из дерева мы на самом деле использовали?

Порядок листьев и глубины узлов. Сегодня оставим только их — два массива. А потом сожмём и это: индекс, который меньше самого текста, и в котором всё равно можно искать.

Пример прежний: $T = \mathrm{abaabaabacb}$, $P = \mathrm{abaab}$, вхождения 0 и 3. Для BWT — `mississippi` из старой лекции курса 2009 года.

План дня

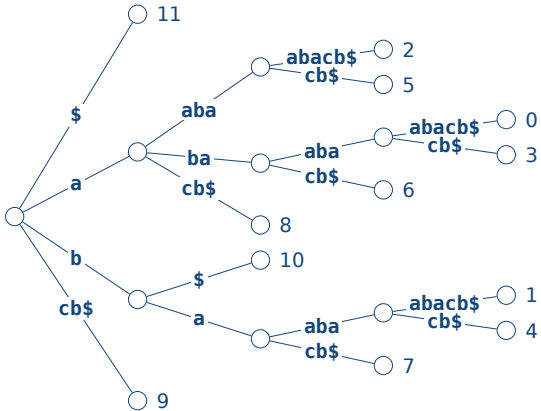
1. Повторение: суффиксное дерево нашего текста, что оно умеет и что стоит.
2. Суффиксный массив: порядок листьев; поиск двумя бинарными поисками; построение удвоением с сортировкой подсчётом.
3. LCP: глубины узлов; алгоритм Касаи; дерево, спрятанное в массиве.
4. Применения на массивах: общая подстрока и MUM двух геномов.
5. Измерения: четыре построителя, счётчики, окупаемость, память; MUMmer 4.
6. BWT и FM-индекс: обратный поиск по сжатому тексту; Bowtie и BWA.

Перерыв — после LCP или перед измерениями, по темпу.

Повторение: суффиксное дерево

Что мы построили в прошлый раз и что оно стоит

Суффиксное дерево abaabaabacb$



Все суффиксы $T\$$ в одном сжатом trie: $n+1$ листьев с началами суффиксов, внутренние узлы ветвятся, метки рёбер — пары индексов в тексте. Узлов не больше $2n+1$.

Вопрос залу

Где в этом дереве образец abaab? И что мы читаем, чтобы его найти?

Подсказка: спуск от корня по буквам образца; где он кончится?

Ответ: спуск за $O(m)$, ответ — листья под точкой

Что умеет дерево	Как	Цена
все вхождения P	спуск по P до *точки остановки* (узла или места внутри ребра); все листья в поддереве под ней	$O(m + \mathrm{occ})$
число вхождений	число листьев в том же поддереве (заранее посчитано)	$O(m)$
самый длинный повтор	самый глубокий внутренний узел: abaaba	$O(n)$
общая подстрока двух текстов	обобщённое дерево, самый глубокий узел с листьями обоих	$O(n_1 + n_2)$
MUM	узел ровно с двумя листьями из разных текстов, проверка слева	$O(n_1 + n_2)$

Образец abaab кончился внутри ребра aba; под этой точкой листья 0 и 3 — это и есть ответ. Всё это — вопросы к *форме* дерева: какие листья под каким узлом и как глубок узел.

Недостатки дерева

Что хранит узел дерева	Байт (64-битные указатели)
указатели на детей: массив на σ символов или список	8σ либо 8 на ребёнка плюс ключ
метка ребра: два индекса	8
суффиксная ссылка	8
номер листа / счётчик листьев	4–8

До $2n$ узлов — десятки байт на букву текста. MUMmer 1999: **37 байт на букву**; MUMmer 3: 32-битное дерево, предел около 500 млн букв опорного генома. Геном человека — 3,1 млрд букв.

И узлы разбросаны по памяти: каждый шаг спуска — промах кэша.

Delcher et al. 1999; Marçais et al. 2018; оценка узла — учебная арифметика, не замер.

Что мы на самом деле использовали

Порядок листьев. Обойдём детей каждого узла по первому символу ребра и выпишем номера листьев:

$$2,\ 5,\ 0,\ 3,\ 6,\ 8,\ 10,\ 1,\ 4,\ 7,\ 9.$$

Это суффиксы T в **лексикографическом порядке**; номер суффикса в этом списке назовём его **рангом**. Все вхождения P — листья одного поддерева — стоят *подряд*.

Глубины узлов. Внутренний узел над листьями $l..r$ с длиной пути d означает: у всех этих суффиксов общий префикс длины d. Достаточно знать d для *соседних* листьев — остальное восстанавливается.

Два массива: **суффиксный массив** SA — порядок, **LCP** — глубины. Без единого указателя.

Суффиксный массив

Порядок суффиксов вместо дерева

Суффиксный массив: позиции суффиксов в отсортированном порядке

r	$SA[r]$	суффикс
0	2	**aabaabacb$**
1	5	**aabacb$**
2	0	**abaabaabacb$**
3	3	**abaabacb$**
4	6	**abacb$**
5	8	**acb$**
6	10	**b$**
7	1	**baabaabacb$**
8	4	**baabacb$**
9	7	**bacb$**
10	9	**cb$**

Ранг суффикса — его индекс в отсортированном списке всех суффиксов, от 0 до $n-1$ (не путать с позицией i в тексте). $SA[r]$ — позиция в тексте суффикса с рангом r.

Массив хранит **позиции**, не копии суффиксов: n чисел; в нашем 32-битном расчёте — по 4 байта. Текст нужен по-прежнему.

Суффиксы с префиксом abaab — ранги 2 и 3 (выделены): $SA[2] = 0$, $SA[3] = 3$.

Manber, Myers, *Suffix arrays: a new method for on-line string searches*, SIAM J. Comput. 1993; старая лекция курса, 20.11.2009.

Все суффиксы с данным префиксом стоят подряд

Если два суффикса начинаются с P, то и все суффиксы между ними начинаются с P: строка без этого префикса на первом отличающемся символе была бы либо меньше обоих, либо больше обоих.

Значит, ответ на «где P» — **интервал рангов** $[L, R)$, и его ищут двумя бинарными поисками: L — первый ранг, чей суффикс не меньше P (сравниваем не дальше m символов); R — первый ранг, чей суффикс строго больше P. Вхождения — $SA[L..R)$, их число — $R - L$. Для abaab: $[2, 4)$, позиции 0 и 3.

Цена: $O(\log n)$ шагов по m символов — $O(m \log n)$ плюс occ.

Старая лекция курса, 20.11.2009, «Поиск образца». Ускорение до $O(m + \log n)$ с помощью LCP — Manber–Myers §3; в заметках.

Почему нельзя просто написать `sort(suffixes)`?

Отсортировать n позиций обычной сортировкой, сравнивая суффиксы посимвольно, как строки. Сколько это стоит на тексте из одной буквы a^n?

Ответ: длинные общие префиксы перечитываются

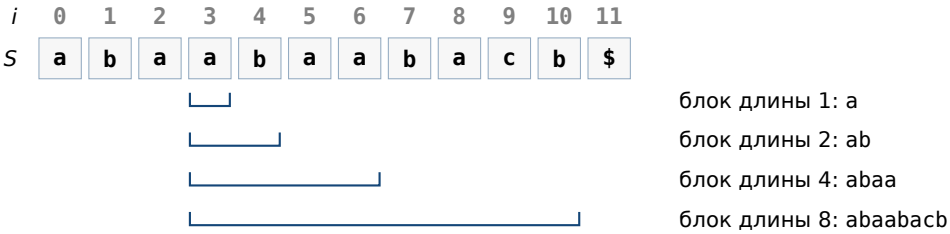
На a^n одно сравнение может прочитать $\Theta(n)$ символов, сравнений $O(n \log n)$: в худшем случае $O(n^2 \log n)$. Нужно перестать перечитывать общие префиксы.

Идея удвоения. Не сравнивать суффиксы целиком, а упорядочить сначала все куски длины 1, потом 2, 4, 8, …Кусок вдвое длиннее — это два коротких куска подряд, а их порядок уже известен.

Чтобы у каждой позиции был кусок любой длины, замкнём строку в кольцо: $S = T\$$, читаем по кругу. Терминатор \$ — самая маленькая буква, поэтому порядок кусков, дошедших до \$, совпадает с порядком суффиксов.

Manber, Myers 1993, §3; вариант с циклическими сдвигами — по плану курса.

Блок: кусок строки длины k, читаемый по кругу



Блок длины k с началом i — подстрока $S[i..i+k)$; если она выходит за конец, продолжаем с начала. Блок длины $2k$ с началом i — это блок длины k с началом i и сразу за ним блок длины k с началом $i+k$.

Суффикс $T[i..)$ — это блок с началом i, дочитанный до \$; длины $1, 2, 4, \ldots$ покрывают его за $\lceil \log_2 n \rceil$ шагов.

Ранг блока: его номер среди различных блоков той же длины

i	0	1	2	3	4	5	6	7	8	9	10	11
S	a	b	a	a	b	a	a	b	a	c	b	$
ранг буквы	1	2	1	1	2	1	1	2	1	3	2	0
пара $(i, i{+}1)$	12	21	11	12	21	11	12	21	13	32	20	01
ранг блока 2	2	5	1	2	5	1	2	5	3	6	4	0

Длина 1: ранги букв — \$$= 0$, a$= 1$, b$= 2$, c$= 3$ (сортировка подсчётом букв).

Длина 2: блок с началом i — пара $(\text{ранг}[i],\ \text{ранг}[i+1])$. Сортируем *пары чисел*, нумеруем различные: 7 рангов. Равные блоки получают один ранг: ab у позиций 0, 3, 6 (оранжевые).

Раунды удвоения на нашем тексте

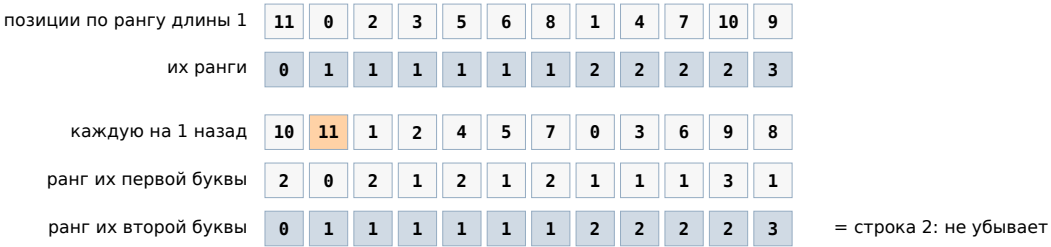
i	0	1	2	3	4	5	6	7	8	9	10	11
S	a	b	a	a	b	a	a	b	a	c	b	$
ранг блока длины 1	1	2	1	1	2	1	1	2	1	3	2	0
ранг блока длины 2	2	5	1	2	5	1	2	5	3	6	4	0
ранг блока длины 4	2	6	1	2	6	1	3	7	4	8	5	0
ранг блока длины 8	3	8	1	4	9	2	5	10	6	11	7	0

Каждая строка — ранги блоков вдвое длиннее, посчитанные по парам рангов предыдущей строки. Различных рангов: 4, 7, 9, 12. Когда все 12 различны, блоки различимы — порядок рангов и есть порядок суффиксов.

demo/traces.txt, раунды удвоения.

Один раунд, шаг 1: порядок по второй компоненте достаётся даром

Нужно отсортировать пары $(\text{ранг}[i],\ \text{ранг}[i+k])$; здесь $k = 1$. Возьмём позиции в порядке их рангов (строка 1) и сдвинем *каждую* на k назад (строка 3): вторая компонента позиции $i - k$ — это ранг позиции i, а они идут по возрастанию.



Позиция 0 после сдвига стала 11: строка — кольцо, $0 - 1 \equiv 11 \pmod{12}$.
Строка 3 — все позиции, упорядоченные по второй компоненте.

Один раунд, шаг 2: стабильная сортировка по первой компоненте

Список после сдвига уже упорядочен по второй компоненте. Досортируем его по первой — рангу самой позиции — **стабильно**, подсчётом, $O(n)$: внутри одного ранга (оранжевые) порядок остаётся прежним, то есть по второй компоненте.

список после сдвига	10	11	1	2	4	5	7	0	3	6	9	8
ранг первой буквы	2	0	2	1	2	1	2	1	1	1	3	1
подсчётом по первой	11	2	5	0	3	6	8	10	1	4	7	9
пары	01	11	11	12	12	12	13	20	21	21	21	32
ранг длины 2	0	1	1	2	2	2	3	4	5	5	5	6

Результат — порядок пар, то есть блоков длины 2; ранг длины 2 растёт там, где пара изменилась.

Удвоение: псевдокод

```
1  # S = T + '$', N = |S|; cyclic shifts, '$' is the
       smallest symbol
2  rank[i] = rank of letter S[i]                # blocks of
       length 1: $=0, a=1, ...
3  k = 1
4  while number of distinct ranks < N:
5      # order of blocks of length 2k = order of pairs
        (rank[i], rank[i+k])
6      shifted = [ (p − k) mod N  for p in order ]   #
        already sorted by 2nd component
7      order = stable counting sort of shifted by rank[.]
         # now by 1st component
8      rank = new ranks: +1 when the pair changes along
        order
9      k = 2k
10 SA = order without the position of '$'
```

Учебный псевдокод; сортировка подсчётом — из лекции 1.

- Строка 2: ранги букв. Строка 7: сдвиг начал на k назад — порядок по второй компоненте.
- Строка 8: стабильная сортировка подсчётом по первой компоненте.
- Строка 9: новый ранг там, где пара изменилась.
- $O(\log n)$ раундов по $O(n)$: $O(n \log n)$.

Настоящий код: один раунд на C++

```
43      for (int j = 0; j < n; ++j)
44          shifted[j] = (order[j] - width + n) % n;
45      std::fill(count.begin(), count.begin()+classes, 0);
46      for (int i : shifted) ++count[rank[i]];
47      for (int c = 1; c < classes; ++c) count[c] +=
    count[c-1];
48      for (int j = n-1; j >= 0; --j) {
49          int i = shifted[j];
50          order[--count[rank[i]]] = i;  // Stable counting
    sort.
51      }
52      next[order[0]] = 0;
53      int new_classes = 1;
54      for (int j = 1; j < n; ++j) {
55          int a = order[j-1], b = order[j];
56          new_classes += std::pair{rank[a],
    rank[(a+width)%n]}
57                         != std::pair{rank[b],
    rank[(b+width)%n]};
58          next[b] = new_classes-1;
59      }
```

- rank — ранг блока. 43–44: сдвиг начал на `width` назад — порядок по второй компоненте унаследован.
- 45–51: подсчёт, префиксные суммы, расстановка *с конца* — стабильно.
- 53–59: ранг растёт, когда пара изменилась. Цикл (строка 42) кончается, когда различных рангов n.

Учебная реализация курса, подготовлена с помощью ИИ: 2026/drafts/suffix-indexes/demo/bench.cpp:43–59; та же функция измеряется в блоке «Измерения».

Цена построения

Метод	Один раунд	Всё построение
наивно: `sort` с посимвольным компаратором	—	до $O(n^2 \log n)$
удвоение + сортировка сравнением (`std::sort` пар)	$O(n \log n)$	$O(n \log^2 n)$
удвоение + сортировка подсчётом	$O(n)$	$O(n \log n)$
SA-IS (Nong, Zhang, Chan, 2009), induced sorting	—	$O(n)$
через суффиксное дерево: Укконен + обход листьев	—	$O(n)$, но память дерева

Линейно можно и через дерево: построить его Укконеном и обойти листья по порядку — $O(n)$, зато десятки байт на букву на время построения. Прямые линейные построители (SA-IS в ACL и Go, libsais, libdivsufsort в MUMmer 4) обходятся без дерева; идея — на следующем кадре. Линейная оценка не говорит, кто быстрее на конкретном входе — измерим через час.

Nong, Zhang, Chan, *Linear suffix array construction by almost pure induced-sorting*, DCC 2009.

SA-IS: идея линейного построения в двух словах

дополнительно

Разделим суффиксы на два типа: **S**, если суффикс меньше следующего ($T[i..) < T[i+1..)$), и **L**, если больше. Тип определяется за один проход справа налево: при равных буквах тип наследуется от соседа справа.

позиция	0	1	2	3	4	5	6
banana$	b	a	n	a	n	a	$
тип	L	S	L	S	L	L	S
LMS		*		*			*

LMS-позиции — S-суффиксы сразу после L-суффикса; их не больше $n/2$. Зная порядок LMS-суффиксов, порядок остальных *выводят* двумя проходами по корзинам первой буквы: L слева направо, S справа налево. Порядок LMS получают рекурсивно на строке вдвое короче: $T(n) = T(n/2) + O(n) = O(n)$.

Nong, Zhang, Chan, DCC 2009; реализация — `sa_is` в AtCoder Library, Go `index/suffixarray`, libsais.

LCP

Глубины узлов: дерево, спрятанное в массиве

LCP: общий префикс соседних суффиксов

r	$SA[r]$	суффикс	$LCP[r]$
0	2	**aabaabacb$**	0
1	5	**aabacb$**	4
2	0	**abaabaabacb$**	1
3	3	**abaabacb$**	6
4	6	**abacb$**	3
5	8	**acb$**	1
6	10	**b$**	0
7	1	**baabaabacb$**	1
8	4	**baabacb$**	5
9	7	**bacb$**	2
10	9	**cb$**	0

$LCP[r]$ — длина наибольшего общего префикса суффиксов с рангами $r-1$ и r; $LCP[0]=0$. Ранги 2 и 3: abaabaabacb и abaabacb — общий префикс abaaba, $LCP[3]=6$. LCP соседей достаточно: общий префикс *любых* двух суффиксов с рангами $l<r$ равен $\min LCP[l+1..r]$.

Старая лекция курса, 20.11.2009, «Значения Lcp»; Kasai et al. 2001.

Как посчитать LCP за $O(n)$: алгоритм Касаи

Наивно — $\Theta(n^2)$ в худшем случае. **Наблюдение.** Пусть суффикс i и его сосед слева j имеют общий префикс длины h. Отбросим по первой букве: суффиксы $i+1$ и $j+1$ имеют общий префикс длины $h-1$ — и сосед слева у $i+1$ совпадает с ним *не меньше*, чем на $h-1$.



Суффиксы 3 и 0 совпадают на 6; значит, 4 и 1 — не меньше чем на 5: сравнение для суффикса 4 начинается с шестого символа.

Касаи: псевдокод и трасса

```
1 rank[SA[r]] = r  for all r            #
      ISA
2 h = 0
3 for i = 0 .. n−1:                     #
      suffixes in text order
4     if rank[i] == 0: h = 0; continue
5     j = SA[rank[i] − 1]               #
      lexicographic predecessor of suffix i
6     while T[i+h] == T[j+h]: h += 1    #
      start from the inherited h, not from 0
7     LCP[rank[i]] = h
8     if h > 0: h −= 1                  #
      next suffix loses at most one
      character
```

i	$ISA[i]$	сосед j	старт h	LCP
0	2	5	0	1
1	7	10	0	1
2	0	—	—	0
3	3	0	0	6
4	8	1	5	5
5	1	2	4	4
6	4	3	3	3
7	9	4	2	2
8	5	6	1	1
9	10	7	0	0
10	6	8	0	0

$ISA[SA[r]] = r$. **Почему** $O(n)$**:** h растёт лишь на успешных сравнениях ($\le n$), а падает не более чем на 1 за шаг. Значит успешных сравнений $O(n)$; неудачных — не более одной на позицию.

Kasai, Lee, Arimura, Arikawa, Park, *Linear-time longest-common-prefix computation in suffix arrays and its applications*, CPM 2001. Трасса: demo/traces.txt.

Настоящий код: LCP из AtCoder Library

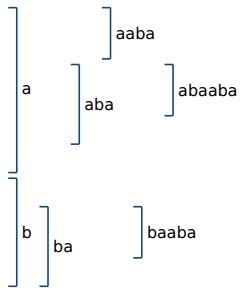
```
222     std::vector<int> lcp(n - 1);
223     int h = 0;
224     for (int i = 0; i < n; i++) {
225         if (h > 0) h--;
226         if (rnk[i] == 0) continue;
227         int j = sa[rnk[i] - 1];
228         for (; j + h < n && i + h < n; h++) {
229             if (s[j + h] != s[i + h]) break;
230         }
231         lcp[rnk[i] - 1] = h;
232     }
233     return lcp;
234 }
```

- **AtCoder Library** — библиотека для соревнований AtCoder, CC0; `lcp_array` возвращает $n - 1$ значений (без нашего нуля в начале).
- Строки 224–226: rnk — наш *ISA*; *h* уменьшается в начале следующей итерации.
- Строки 228–230: сравнение продолжается с сохранённого *h*.

atcoder/ac-library v1.5.1 · CC0 atcoder/string.hpp:222–234 (копия в lectures/20261014/refs)

Дерево, спрятанное в массиве

r	SA	суффикс	LCP
0	2	**aabaabacb$**	0
1	5	**aabacb$**	4
2	0	**abaabaabacb$**	1
3	3	**abaabacb$**	6
4	6	**abacb$**	3
5	8	**acb$**	1
6	10	**b$**	0
7	1	**baabaabacb$**	1
8	4	**baabacb$**	5
9	7	**bacb$**	2
10	9	**cb$**	0



Скобка — отрезок рангов $[l, r]$, внутри которого все $LCP \geq d$, а сразу за краями меньше d: у всех этих суффиксов общий префикс длины d (подпись у скобки). Это ровно **внутренний узел** дерева глубины d с листьями $SA[l..r]$: abaaba над 0 и 3, а над шестью суффиксами.

Повторы и число различных подстрок

Самый длинный повтор — самый глубокий узел — **максимум** *LCP*: 6, abaaba, соседи $SA[2] = 0$ и $SA[3] = 3$.

Число различных подстрок. Каждая подстрока — префикс какого-то суффикса. Суффикс длины ℓ имеет ℓ непустых префиксов; всего $1 + 2 + \cdots + n = n(n+1)/2$. Но идём по суффиксам в отсортированном порядке: первые $LCP[r]$ префиксов суффикса с рангом r уже были у суффикса с рангом $r - 1$ — это повторы, вычитаем:

$$\frac{n(n+1)}{2} - \sum_r LCP[r] = 66 - 23 = 43.$$

Проверено перебором: 43. Для aaaa: $10 - 6 = 4$ (a, aa, aaa, aaaa).

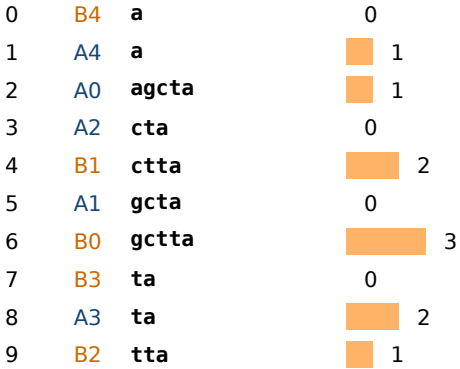
Число вхождений P — длина интервала $R - L$, без обхода поддерева.

demo/traces.txt; лекция 20.11.2009.

Применения на массивах

Общая подстрока и MUM без дерева

Обобщённый суффиксный массив двух геномов



Напоминание: **MUM** (maximal unique match) — кусок, который встречается ровно один раз в A и ровно один раз в B и не продлевается ни влево, ни вправо; якоря выравнивания в MUMmer.

$A = \texttt{agcta}$, $B = \texttt{gctta}$ — те же «геномы», что в прошлый раз. Суффиксы обеих строк в одном отсортированном массиве; у каждого — метка строки (A синим, B оранжевым) и позиция.

LCP считается между соседями, как обычно; суффиксы обрезаны своими терминаторами.

Там, где в дереве мы смотрели «под каким узлом листья обоих цветов», здесь смотрим на **соседей разных цветов**.

Один узел дерева — один LCP-интервал

Рассмотрим `gct` в нашем обобщённом массиве:

Ранг	Суффикс	*LCP*	Что это значит
4	B1: `cta`	2	перед интервалом
5	A1: `gcta`	0	левый край
6	B0: `gcta`	3	общий префикс `gct`
7	B3: `ta`	0	правый край

Внутри рангов $[5, 6]$ общий префикс длины 3, а снаружи $LCP < 3$: третьего суффикса с `gct` нет. Это ровно узел `gct` с двумя листьями из дерева лекции 6. Справа совпадение не продлить: после него `a` и `t`.

Достаточно ли этого для MUM? Что ещё может пойти не так?

Общая подстрока и MUM по массиву

Наибольшая общая подстрока. Самый глубокий узел с листьями обоих цветов $\rightarrow$ максимум $LCP[r]$ среди пар соседей *разного* цвета: ранги 5 и 6, $LCP = 3$, gct. Один проход.

MUM — узел ровно с двумя листьями из разных строк $\rightarrow$ пара соседей разного цвета с $LCP[r] = \ell$, у которой *оба внешних соседа* имеют $LCP < \ell$ (строка встречается ровно по разу), плюс проверка символов слева.

Соседи	общий префикс	снаружи *LCP*	слева	
3, 4 (A2, B1)	ct, 2	0 и 0	g / g	продлевается: не MUM
5, 6 (A1, B0)	gct, 3	0 и 0	a / начало	**MUM**
7, 8 (B3, A3)	ta, 2	0 и 1	t / c	**MUM**

Тот же ответ, что по дереву, за те же $O(n_1 + n_2)$ — по двум плотным массивам.

demo/traces.txt, обобщённый массив; лекция 6, кадр «Ответ: общая подстрока и MUM — это узлы».

Что подешевело, что подорожало

	Дерево	$SA + LCP$
память	десятки байт на букву, указатели	8–12 байт на индексы + 1 за текст
построение	$O(n)$ Укконен	$O(n \log n)$ удвоение; $O(n)$ SA-IS
поиск образца	$O(m + \mathrm{occ})$	$O(m \log n + \mathrm{occ})$; $O(m + \log n)$ с доп. подготовкой
повторы, LCS, MUM	обход дерева	проход по *LCP*
локальность	узлы разбросаны	последовательные массивы, бинарный поиск
текст приходит по частям	онлайн	перестроить

Проиграли логарифм в поиске и онлайн-построение; выиграли память и локальность. Что перевешивает — измерим.

Измерения

Четыре построителя, счётчики, окупаемость, память

Что и как сравниваем

- **Наивный:** `std::sort` позиций с посимвольным компаратором.
- **Удвоение + `std::sort` пар:** то же удвоение (ранги блоков длины 1, 2, 4, …), но пары на каждом раунде сортирует `std::sort`; `sa_doubling` из AtCoder Library, $O(n \log^2 n)$.
- **Удвоение + подсчёт:** наш код из раздела про массив, $O(n \log n)$, с ранней остановкой, когда все ранги различны.
- **SA-IS:** публичный `suffix_array` из AtCoder Library, $O(n)$.

Два текста: случайный над `acgt` и периодический abaaba…; $n = 8192$ и $65\,536$. Семь повторов, медиана; 300 входов сверены между построителями до замера.

Стенд: Ryzen AI MAX+ 395, CPU 1, GCC 13.3 -03; 2026/drafts/suffix-indexes: demo/bench.cpp, benchmark-metadata.json; numbers.md, S02.

Построение: повторяющийся текст меняет картину

Медиана, мс	случайный, 8192	периодический, 8192	случайный, 65 536	периодический, 65 536
наивный `sort` суффиксов	0,81	**129,9**	—	—
удвоение, пары через `std::sort`	1,16	1,27	14,08	15,30
удвоение, пары подсчётом	**0,17**	0,49	**1,50**	4,86
SA-IS (ACL)	0,27	**0,06**	2,36	**0,47**

Наивная сортировка на случайном тексте быстрее удвоения со `std::sort`: суффиксы различаются во второй-третьей букве. На периодическом она читает длинные префиксы — в 160 раз медленнее.

numbers.md черновика, S02; сырые серии — demo/benchmark-samples.csv.

Улучшение, которое не всегда выиграло

Заменить $O(n \log n)$ на $O(n)$ — разумная идея. На нашем случайном тексте $n = 65\,536$ SA-IS оказался в 1,6 раза *медленнее* удвоения с подсчётом; на периодическом — в 10 раз быстрее.

Почему: удвоение с подсчётом останавливается, как только все ранги различны — на случайном тексте после двух-трёх раундов. На периодическом тексте ранги склеены до самого конца, и линейный алгоритм выигрывает.

Гарантия роста и время конкретного запуска — разные утверждения. Та же мораль, что у AG и Two-Way.

numbers.md черновика, S02.

Счётчики: где исчезла работа

На одно построение, случайный текст $n = 65\,536$	удвоение + `std::sort`	удвоение + подсчёт
инструкции, млн	242,2	15,5
такты, млн	72,2	7,3
ветвления, млн	53,5	1,3
промахи предсказателя, тыс.	717,0	0,4

В 16 раз меньше инструкций и в 1600 раз меньше промахов: сортировка подсчётом не ветвится по данным, а ранняя остановка убирает лишние раунды. Счётчики не разделяют эти два вклада.

`perf stat`, события `:u`, три процесса по 100 построений, медиана /100; demo/perf-*.txt; numbers.md, S04.

Запросы: когда индекс окупается

4096 запросов «есть ли», образцы по 32 байта, мс на пакет	случайный, 65 536	периодический, 65 536
`std::string::find` без индекса	168,1	248,7
бинарный поиск по *SA*, индекс готов	**0,63**	**0,15**

Половина образцов есть в тексте, у другой половины последний байт заменён нулём — такие запросы заставляют `find` просканировать весь текст.

Окупаемость: $B + Q\,t_{\text{index}} < Q\,t_{\text{scan}}$. Построение 1,5 мс, запрос 0,15 мкс против 41 мкс: индекс окупается после **37 запросов**. Один запрос — сканируй.

numbers.md черновика, S03; оценка по пакетным средним, не гарантия для любого Q.

Память и MUMmer 4

Учебный расчёт с 32-битными индексами: *SA* $4n$ + *ISA* $4n$ + *LCP* $4n$ + текст $n = 13n$ байт. Дерево MUMmer 1999 — 37 байт на букву; MUMmer 4 (2018): около 15 байт на букву опорного генома.

MUMmer 3 упирался в 32-битное дерево (около 500 млн букв). MUMmer 4 заменил его 48-битным суффиксным массивом с *ISA* и *LCP*: человек против шимпанзе, $3{,}09$ и $3{,}31$ млрд букв, 3 ч 6 мин на 32 потоках, 66 ГБ. Индекс строится один раз и переиспользуется.

Авторы честно пишут: одновременно изменились параллельная обработка и конвейер, так что общее ускорение нельзя приписать одному массиву. Но предел размера снял именно массив.

Marçais et al. 2018; numbers.md черновика, S06. Расчёт $13n$ — арифметика представления, не замер.

BWT и FM-индекс

Индекс меньше текста, в котором всё равно можно искать

Можно ли искать, не храня ни текста, ни массива?

Массив — $4n$ байт плюс текст. Геном человека: 3,1 млрд букв по 2 бита — 0,8 ГБ текста и 12 ГБ массива.

Что останется, если выкинуть и указатели, и массив, и даже сам текст?

Ответ 1994–2000 годов: **преобразование Барроуза–Уилера** и **FM-индекс** Феррагины и Манзини. Строка той же длины, что текст, хорошо сжимается, и по ней можно искать за $O(m)$ шагов, не распаковывая.

Это то, что внутри `bzip2` (сжатие) и внутри Bowtie и BWA (выравнивание прочтений на геном).

Burrows, Wheeler, *A block-sorting lossless data compression algorithm*, DEC SRC 124, 1994; Ferragina, Manzini, *Opportunistic data structures with applications*, FOCS 2000; старая лекция курса, 04.12.2009.

BWT: последний столбец отсортированных вращений

	SA	F											L
0	11	$	m	i	s	s	i	s	s	i	p	p	i
1	10	i	$	m	i	s	s	i	s	s	i	p	p
2	7	i	p	p	i	$	m	i	s	s	i	s	s
3	4	i	s	s	i	p	p	i	$	m	i	s	s
4	1	i	s	s	i	s	s	i	p	p	i	$	m
5	0	m	i	s	s	i	s	s	i	p	p	i	$
6	9	p	i	$	m	i	s	s	i	s	s	i	p
7	8	p	p	i	$	m	i	s	s	i	s	s	i
8	6	s	i	p	p	i	$	m	i	s	s	i	s
9	3	s	i	s	s	i	p	p	i	$	m	i	s
10	5	s	s	i	p	p	i	$	m	i	s	s	i
11	2	s	s	i	s	s	i	p	p	i	$	m	i

Все циклические сдвиги `mississippi$`, отсортированные. Первый столбец F — буквы по порядку; последний L — **BWT**: `ipssm$pissii`.

Строка r начинается с суффикса $SA[r]$, а кончается буквой перед ним: $L[r] = T[(SA[r] - 1) \bmod N]$, $N = |T|$. BWT — это столбец, который *даром* получается из суффиксного массива.

Строки отсортированы по тому, что идёт *после* буквы L; соседние строки начинаются одинаково, а перед одинаковым началом часто стоит одна буква: перед `si…` всегда `s`, перед `ssi…` всегда `i`. Серии в L (`ss`, `ii`) и сжимает `bzip2`.

Вопрос залу

В столбце *L* четыре буквы s, в столбце *F* тоже четыре. Это одни и те же четыре буквы текста. Как узнать, какая s из *L* стоит в какой строке *F*?

Подсказка: строка, кончающаяся на s, — вращение текста; сдвиньте её на одну букву вправо — получится строка, *начинающаяся* на s.

Ответ: k-я буква c в L — это k-я буква c в F

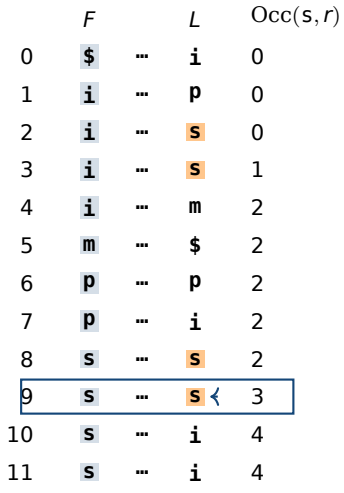
Строки, кончающиеся на c, после сдвига на одну букву становятся строками, *начинающимися* на c, в том же взаимном порядке: их упорядочили те же хвосты. Отсюда отображение **LF**: строка r с $L[r] = c$ переходит в строку $C[c] + \mathrm{Occ}(c, r)$, где $C[c]$ — число букв меньше c, $\mathrm{Occ}(c, r)$ — число c в $L[0..r)$. Для `mississippi$`: $C = \{\${:}0,\ \mathtt{i}{:}1,\ \mathtt{m}{:}5,\ \mathtt{p}{:}6,\ \mathtt{s}{:}8\}$.

```
1 # L = BWT, F = sorted L, C[c] = number of
      symbols < c in the text
2 LF(r) = C[L[r]] + Occ(L[r], r)        #
      the k-th c in L is the k-th c in F
3
4 restore(L, first):                    #
      first is the row with SA[first] = 0
5     r = first                         #
      the original rotation, not just row 0
6     for n times:
7         output L[r]
8         r = LF(r)
9     reverse the output
```

- Строка 2: LF ведёт от буквы к *предыдущей* букве текста.
- Строки 4–8: `first` — строка исходного вращения ($SA = 0$), её номер храним отдельно. За n шагов LF восстанавливаем текст: нужны только L, C и Occ.

Старая лекция курса, 04.12.2009, «Восстановление строки в один проход»; Burrows–Wheeler 1994.

$\mathrm{Occ}(c, r)$: сколько букв c стоит в L выше строки r

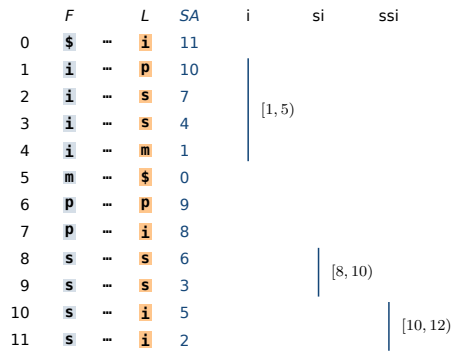


Чтобы перейти от буквы в L к той же букве в F, надо знать, *какая по счёту* это буква среди одинаковых. Для буквы c в строке r это число букв c в $L[0..r)$ — $\mathrm{Occ}(c, r)$.

В строке 9 стоит s; выше неё три s: $\mathrm{Occ}(\mathrm{s}, 9) = 3$, это четвёртая s. В F буквы s начинаются со строки $C[\mathrm{s}] = 8$, четвёртая — строка $8 + 3 = 11$. Это и есть $LF(9)$.

Обратному поиску нужно то же самое для границ интервала: сколько c выше строки sp и сколько выше ep — столько строк с c лежит внутри интервала и столько окажется в начале нового.

Обратный поиск: интервал растёт влево



Ищем ssi *с конца*. Строки с i: $[1,5)$, это C[i]..C[m]. Строки с si: те из $[1,5)$, где L = s, после LF — $[8,10)$. Строки с ssi: $[10,12)$; их SA — 5 и 2, оба вхождения.

```
1  backward_search(P):                          #
       P read from its last symbol to the
       first
2      sp, ep = 0, n                            #
       all rows
3      for c in reversed(P):
4          sp = C[c] + Occ(c, sp)
5          ep = C[c] + Occ(c, ep)
6          if sp >= ep: return empty
7      return [sp, ep)                          #
       rows of SA whose suffix starts with
       P
```

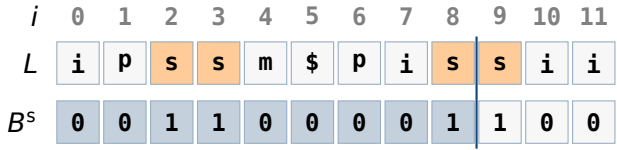
Для позиций нужна выборка значений SA (sampled SA); для ответа «сколько» она не нужна.

Старая лекция курса, 04.12.2009, «Алгоритм обратного поиска»; Ferragina, Manzini 2000. Трасса: demo/traces.txt.

Вся цена — в $\mathrm{Occ}(c, i)$: битовый вектор на каждую букву

$\mathrm{Occ}(c, i)$ — сколько букв c среди первых i букв столбца L. Наивно — проход по L, $O(n)$ на каждую букву образца.

Заведём для каждой буквы c **битовый вектор** B^c длины n: $B^c[j] = 1$, если $L[j] = c$. Тогда $\mathrm{Occ}(c, i)$ — число единиц среди первых i битов B^c: это операция $\mathrm{rank}_1(B^c, i)$.

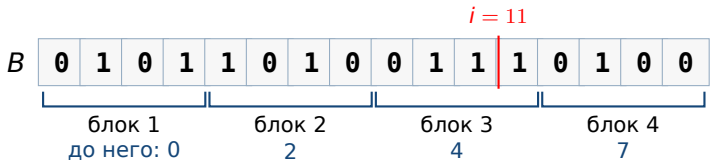


$\mathrm{Occ}(\mathtt{s}, 9) = \mathrm{rank}_1(B^{\mathtt{s}}, 9) = 3$. Задача свелась к одной: быстро считать единицы в префиксе битового вектора.

Старая лекция курса, 04.12.2009, «Рассчёт Occ»; Ferragina, Manzini 2000.

rank за $O(1)$: сохранить суммы по блокам

Разобьём B на блоки по b бит и для каждой границы блока запомним число единиц *до* неё. Тогда $\mathrm{rank}_1(B, i)$ = сумма до начала блока, в который попал i, плюс единицы внутри блока до i.

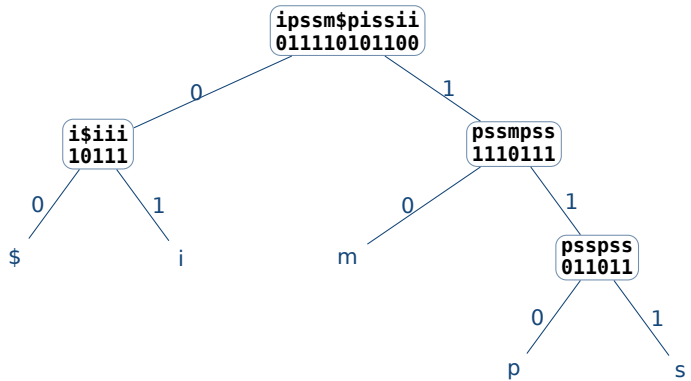


$\mathrm{rank}_1(B, 11)$ = единицы до блока 3 (4) + единицы в `011` внутри блока 3 (2) $= 6$. Внутри блока — одна инструкция (`popcount`) или маленькая таблица. Память сверх B: число на блок, при $b = \log n$ это $o(n)$ бит. Обратный поиск: $O(m)$ операций rank, **не зависит от длины текста**.

Старая лекция курса, 04.12.2009, «Основная идея», «Пример»; Jacobson 1989.

Вейвлет-дерево: один вектор на уровень, не на букву

Вектор B^c на каждую из σ букв алфавита — σn бит. **Вейвлет-дерево** хранит то же в $n \log_2 \sigma$ бит. В корне — вся строка L и один бит на позицию: 0, если буква из младшей половины алфавита $\{\$, \mathtt{i}\}$, 1 — из старшей $\{\mathtt{m}, \mathtt{p}, \mathtt{s}\}$. Левый ребёнок — буквы с битом 0 по порядку, правый — с битом 1; и так, пока в узле не останется одна буква.



Occ по вейвлет-дереву: один rank на уровень

$\mathrm{Occ}(\mathsf{s}, 9)$ — спуск от корня к листу s, на каждом уровне rank по битам узла:

Узел	биты первых i позиций	единиц	куда
корень, ipssm$pissii, $i = 9$	011110101	6	s в старшей половине: вправо, $i = 6$
pssmpss, $i = 6$	111011	5	s в $\{\mathsf{p}, \mathsf{s}\}$: вправо, $i = 5$
psspss, $i = 5$	01101	3	лист s: $\mathrm{Occ} = 3$

Если бы буква была в младшей половине, шли бы влево с $i -$ единиц — числом нулей.

$\log_2 \sigma$ операций rank на букву образца: запрос $O(m \log \sigma)$. Память: $n \log \sigma$ бит плюс добавка на rank — для ДНК ($\sigma = 4$) около **2 бит на букву**: столько же, сколько сам текст в двухбитной кодировке.

Старая лекция курса, 04.12.2009, «Occ(a,16)»; Grossi, Gupta, Vitter 2003. Путь посчитан tools/arrays.py.

Где это живёт

- **bzip2** (1996): BWT блока, затем move-to-front и Хаффман; сжимает серии одинаковых букв в L.
- **Bowtie** (2009) и **BWA** (2009): FM-индекс генома человека; миллионы коротких прочтений ищутся обратным поиском с допуском ошибок. По данным авторов Bowtie, индекс генома человека занимает около 1,3 ГБ памяти.
- **MUMmer 4** остался на суффиксном массиве: ему нужны LCP и длинные совпадения, а не миллионы коротких запросов.

Одна и та же идея — отсортированные суффиксы — в трёх представлениях: дерево, массив, BWT. Чем меньше памяти, тем дороже каждый шаг и тем меньше задач решается напрямую.

Seward, bzip2; Langmead, Trapnell, Pop, Salzberg, *Ultrafast and memory-efficient alignment of short DNA sequences to the human genome*, Genome Biology 10:R25, 2009; Li, Durbin, *Fast and accurate short read alignment with Burrows–Wheeler transform*, Bioinformatics 2009.

Откуда мы знаем

- Суффиксный массив удвоением сверен с наивной сортировкой; LCP Касаи — с попарным сравнением соседей; число различных подстрок — с перебором (43).
- Восстановление BWT и обратный поиск сверены с исходной строкой и прямым поиском на 300 случайных строках и образцах.
- Все таблицы и рисунки этого дня — вывод одного скрипта (`tools/arrays.py`), журнал — `demo/traces.txt`.
- Измерения — четыре C++ построителя черновика, 300 входов сверены между ними до замера; хеши исходников записаны рядом с числами.

Доказательства говорят про рост, проверки — про правильность на малых входах, замеры — про этот стенд. Три разных вопроса.

Три структуры — один порядок суффиксов

	суффиксное дерево	$SA + LCP$	FM-индекс (BWT)
память на букву	десятки байт	9–13 байт с текстом	$\log\sigma + o(\log\sigma)$ бит; позиции — выборка SA
построение	$O(n)$ Укконен	$O(n\log n)$ удвоение, $O(n)$ SA-IS	из SA
есть ли P	$O(m)$	$O(m\log n)$	$O(m\log\sigma)$
где все P	$+\mathrm{occ}$	$+\mathrm{occ}$	$+\mathrm{occ}\cdot$ цена выборки SA
повторы, LCS, MUM	обход	проход по LCP	отдельные приёмы
кто использует	MUMmer 1–3	MUMmer 4, bsdiff	bzip2, Bowtie, BWA

σ — размер алфавита (4 для ДНК, 256 для байтов). Чем меньше памяти, тем дороже шаг.

Что уносим

- **Из дерева нужны порядок листьев и глубины узлов.** Это *SA* и *LCP*: два плотных массива вместо указателей.
- **Удвоение:** ранги блоков длины k дают ранги блоков длины $2k$ сравнением пар чисел; сортировка подсчётом — раунд за $O(n)$.
- **Касаи:** сдвиг на одну букву теряет не больше одного символа общего префикса — *LCP* за $O(n)$.
- **Линейная оценка не выбирает реализацию:** SA-IS проиграл удвоению на случайном тексте и выиграл на периодическом.
- **BWT — столбец суффиксного массива,** по которому можно идти назад; с C и Occ он ищет за $O(m \log \sigma)$ без текста и полного *SA*.

Спасибо за внимание

https://github.com/alk-2000/da-lectures